

COMP3018 – Human-Robot Interaction

Week 9 – Facial Expression Detection

Semester 2, 2025-2026

The purpose of this workshop is to have work on a dataset of facial expressions and train a neural network model to recognize emotions. This exercise builds on the theoretical knowledge you learned in the module COMP3018.

Exercise 1

Emotion detection is a challenging task, because emotions are subjective. There is no common consensus on how to measure or categorize them. In this exercise we will learn how to make classification over a dataset of facial expressions. We use **tensorflow** **keras** - an open source Python API for deep learning - and we can make use of it to build the CNN model. We will also rely on **numpy** , and **OpenCV** which is a library for computer vision

- Import the libraries **numpy**, **cv2**, and from **tensorflow.keras.preprocessing.image** import the function **ImageDataGenerator**, from **tensorflow.keras.layers** import the functions **Dense**, **Input**, **Dropout**, **Flatten**, **Conv2D** , from **tensorflow.keras.layers** import the functions **BatchNormalization**, **Activation**, **MaxPooling2D**, from **tensorflow.keras.models** import the functions **Model**, **Sequential**, and continue importing as indicated in the image.
- Download the facial expression dataset from Kaggle and check its structure Dataset: www.kaggle.com/datasets/aadityasinghal/facial-expression-dataset.
- Get some information about the train and test folders

```
6 import numpy as np
7 import seaborn as sns
8 import matplotlib.pyplot as plt
9 import utils
10 import os
11 import cv2
12 from tensorflow.keras.preprocessing.image import ImageDataGenerator
13 from tensorflow.keras.layers import Dense, Input, Dropout, Flatten, Conv2D
14 from tensorflow.keras.layers import BatchNormalization, Activation, MaxPooling2D
15 from tensorflow.keras.models import Model, Sequential
16 from tensorflow.keras.optimizers import Adam
17 from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau
18 from tensorflow.keras.utils import plot_model
19 from IPython.display import SVG, Image
20 from livelossplot.inputs.tf_keras import PlotLossesCallback
21 import tensorflow as tf
22 print("Tensorflow version:", tf.__version__)
23
24 PATH_OF_TRAIN_FOLDER = "/Users/amiraly/Desktop/FacesData/train/train/"
25
26 for expression in os.listdir(PATH_OF_TRAIN_FOLDER):
27     print(str(len(os.listdir(PATH_OF_TRAIN_FOLDER + expression))) + " " + expression + "
28           images")
```

- Generate training and testing (validation) batches so that our model could be trained and evaluated/validated on the test data.

```

30 img_size = 48
31 batch_size = 64
32 datagen_train = ImageDataGenerator(horizontal_flip=True)
33 train_generator = datagen_train.flow_from_directory(PATH_OF_TRAIN_FOLDER,
34                                                    target_size=(img_size,img_size),
35                                                    color_mode="grayscale",
36                                                    batch_size=batch_size,
37                                                    class_mode='categorical',
38                                                    shuffle=True)
39 datagen_validation = ImageDataGenerator(horizontal_flip=True)
40 validation_generator = datagen_validation.flow_from_directory(PATH_OF_TRAIN_FOLDER,
41                                                             target_size=(img_size,img_size),
42                                                             color_mode="grayscale",
43                                                             batch_size=batch_size,
44                                                             class_mode='categorical',
45                                                             shuffle=False)

```

- In the above code we have defined the image size to 48 so each image will be reduced to a size of 48x48. After that we have defined the **batch size equal to 64** which means that in each epoch ,i.e., when the model is ran through the training dataset at each cycle , the number of images that will be passed will be 64. Which means that model will take first 64 images for training through first epoch and will continue so on till all the epochs are completed.
- Build the CNN Model as indicated below. First of all we *initialize the CNN model by using sequential() function*. After that we create the **first four layers** of the neural network which are **convolution layers**. **Activation function** is added which does non linear transformation that we do over the input signal. This transformed output is then sent to the next layer of neurons as input. And after that we apply **MaxPolling2D** which take out only the maximum from a pool. We add **dropout** which prevents the model from over-fitting. Then, we add two fully connected layers to the model, a **Batch Normalization** layer, and **activation and dropout** layers.

```

47 # Initialising the CNN
48 model = Sequential()
49 # 1 - Convolution
50 model.add(Conv2D(64,(3,3), padding='same', input_shape=(48, 48,1)))
51 model.add(BatchNormalization())
52 model.add(Activation('relu'))
53 model.add(MaxPooling2D(pool_size=(2, 2)))
54 model.add(Dropout(0.25))
55 # 2nd Convolution layer
56 model.add(Conv2D(128,(5,5), padding='same'))
57 model.add(BatchNormalization())
58 model.add(Activation('relu'))
59 model.add(MaxPooling2D(pool_size=(2, 2)))
60 model.add(Dropout(0.25))
61 # 3rd Convolution layer
62 model.add(Conv2D(512,(3,3), padding='same'))
63 model.add(BatchNormalization())
64 model.add(Activation('relu'))
65 model.add(MaxPooling2D(pool_size=(2, 2)))
66 model.add(Dropout(0.25))

```

```

67 # 4th Convolution layer
68 model.add(Conv2D(512,(3,3), padding='same'))
69 model.add(BatchNormalization())
70 model.add(Activation('relu'))
71 model.add(MaxPooling2D(pool_size=(2, 2)))
72 model.add(Dropout(0.25))
73 # Flattening
74 model.add(Flatten())
75 # Fully connected layer 1st layer
76 model.add(Dense(256))
77 model.add(BatchNormalization())
78 model.add(Activation('relu'))
79 model.add(Dropout(0.25))
80 # Fully connected layer 2nd layer
81 model.add(Dense(512))
82 model.add(BatchNormalization())
83 model.add(Activation('relu'))
84 model.add(Dropout(0.25))
85 model.add(Dense(7, activation='softmax'))
86 opt = Adam(lr=0.0005)
87 model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
88 model.summary()

```

- Train the model on the training dataset and at the same time it validates on the test/validation dataset. After that save the model parameters to a json file.

```

91 plot_model(model, to_file='model.png', show_shapes=True, show_layer_names=True)
92 Image('model.png',width=400, height=200)
93
94
95 epochs = 15
96 steps_per_epoch = train_generator.n//train_generator.batch_size
97 validation_steps = validation_generator.n//validation_generator.batch_size
98 reduce_lr = ReduceLRonPlateau(monitor='val_loss', factor=0.1,
99                               patience=2, min_lr=0.00001, mode='auto')
100 checkpoint = ModelCheckpoint("model_weights.h5", monitor='val_accuracy',
101                              save_weights_only=True, mode='max', verbose=1)
102 callbacks = [PlotLossesCallback(), checkpoint, reduce_lr]
103 history = model.fit(
104     x=train_generator,
105     steps_per_epoch=steps_per_epoch,
106     epochs=epochs,
107     validation_data = validation_generator,
108     validation_steps = validation_steps,
109     callbacks=callbacks
110 )
111
112
113 model_json = model.to_json()
114 model.save_weights('model_weights.h5')
115 with open("model.json", "w") as json_file:
116     json_file.write(model_json)

```